# 四柱汉诺塔

## 一：思想：

### 用以下算法移动盘子（记为FourPegsHanoi）

### （1）将A柱上的n个盘子划分为上下两部分，上方有k个盘子，下方有n-k个盘子

### （2）将A柱上方的k个盘子使用FourPegsHanoi经C,D柱移到B柱

### （3）将A柱下方的n-k个盘子使用ThreePegsHanoi，经C柱移到D柱

### （4）将B柱上的k个盘子使用FourPegsHanoi经A,C柱移到D柱

## 二：递推式

$$F(n) = \min_{0 \le k < n} \{2F(k) + 2^{n-k} - 1\}$$

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<long long> dp;
vector<int> best;
//best数组记录转移位置，dp数组记录最优值

void hanoi3(int n, char a, char b, char c) {
    if (n == 0) return;
    hanoi3(n - 1, a, c, b);
    cout << a << " -> " << c << endl;
```

```
    hanoi3(n - 1, b, a, c);
}

void hanoi4(int n, char a, char b, char c, char d) {
    if (n == 0) return;
    int k = best[n];
    if (k == 0) {
        hanoi3(n, a, b, d);
        return;
    }
    hanoi4(k, a, c, d, b);
    hanoi3(n - k, a, c, d);
    hanoi4(k, b, a, c, d);
}

int main() {
    int n;
    cin >> n;

    dp.assign(n + 1, 0);
    best.assign(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        dp[i] = (1LL << i) - 1;
        best[i] = 0;
        for (int k = 1; k < i; k++) {
            long long cost = 2 * dp[k] + ((1LL << (i - k))
- 1);
            if (cost < dp[i]) {
                dp[i] = cost;
                best[i] = k;
            }
        }
    }

    hanoi4(n, 'A', 'B', 'C', 'D');
```

```
    return 0;
}
```